

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

MURILO NUNES MARÇAL

**INTEGRAÇÃO E ENTREGA CONTÍNUA: COMPARAÇÃO DE
FUNCIONALIDADES ENTRE JENKINS, GITLAB CI E GITHUB ACTIONS**

GUARAPUAVA

2024

MURILO NUNES MARÇAL

**INTEGRAÇÃO E ENTREGA CONTÍNUA: COMPARAÇÃO DE
FUNCIONALIDADES ENTRE JENKINS, GITLAB CI E GITHUB ACTIONS**

**Continuous Integration and Deployment: Comparing Jenkins, GitLab CI, and
GitHub Actions**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção do
título de Tecnólogo em Sitemas para Internet do
Curso Tecnólogo em Sitemas para Internet da
Universidade Tecnológica Federal do Paraná.

Orientador: Hermano Pereira

GUARAPUAVA

2024



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

RESUMO

Com o crescimento da complexidade dos sistemas distribuídos e aplicações modernas, aliada à necessidade de entregas ágeis e confiáveis, surgiu o conceito de *DevOps*, que busca integrar as equipes de desenvolvimento e operações para otimizar a automação, a colaboração e a entrega de *software*. Dentro dessa abordagem, destacam-se as práticas de Integração Contínua (CI) e Entrega Contínua (CD). A CI refere-se ao processo de automação da integração de código de múltiplos desenvolvedores em um repositório compartilhado, garantindo que mudanças frequentes sejam verificadas e testadas continuamente. A CD complementa esse processo, automatizando a entrega do *software* para ambientes de teste e produção, permitindo implantações rápidas e confiáveis. Para que essas práticas sejam eficazes, utilizam-se *pipelines* (ou esteiras de CI/CD), que são conjuntos de processos automatizados responsáveis por compilar, testar e implantar aplicações de maneira sistemática e reproduzível. Este trabalho apresenta um estudo comparativo entre três das principais ferramentas de CI/CD: GitHub Actions, GitLab CI/CD e Jenkins. A metodologia adotada consiste na implementação de *pipelines* CI/CD em cada plataforma, seguidos de uma avaliação baseada em critérios previamente definidos. Para garantir a confiabilidade dos resultados, os testes serão realizados em um ambiente controlado, onde as variáveis de infraestrutura permanecerão constantes. Com isso, este estudo contribuirá para a escolha da ferramenta de CI/CD mais adequada para diferentes cenários, auxiliando equipes de desenvolvimento na automação de seus fluxos de trabalho.

Palavras-chave: entrega contínua; integração contínua; análise de performance; desenvolvimento e operações.

ABSTRACT

With the increasing complexity of software systems and the need for fast and reliable deliveries, the DevOps approach has emerged to integrate development and operations teams, enhancing automation, collaboration, and software delivery. Within this approach, Continuous Integration (CI) and Continuous Delivery (CD) stand out as fundamental practices. Continuous Integration refers to the process of automating the integration of code from multiple developers into a shared repository, ensuring that frequent changes are continuously verified and tested. Continuous Delivery complements this process by automating software deployment to testing and production environments, enabling fast and reliable releases. To support these practices, CI/CD pipelines (also known as build and deployment pipelines) are used. These pipelines consist of automated workflows that systematically compile, test, and deploy applications, ensuring consistency and reproducibility throughout the software development lifecycle. This study presents a comparative analysis of three leading CI/CD tools: GitHub Actions, GitLab CI/CD, and Jenkins. The methodology involves implementing CI/CD pipelines on each platform, followed by an assessment based on predefined criteria. To ensure result consistency and reliability, experiments will be conducted in a controlled environment where infrastructure variables remain constant. This research aims to provide insights into the capabilities of each tool, helping development teams choose the most suitable solution for automating their workflows.

Keywords: continuous integration; continuous delivery; benchmarking; devops.

LISTA DE ABREVIATURAS E SIGLAS

Abreviaturas

CI	Continuous Integration (Integração Contínua)
CD	Continuous Deployment (Entrega Contínua)
DevOps	Development and Operations (Integração entre Desenvolvimento e Operações)
VPS	Virtual Private Server (Servidor Virtual Privado)
YAML	Yet Another Markup Language (mais uma linguagem de marcação)

SUMÁRIO

1	INTRODUÇÃO	6
1.1	Considerações iniciais	7
1.2	Objetivos	8
1.2.1	Objetivo geral	8
1.2.2	Objetivos específicos	8
1.3	Justificativa	8
2	REFERENCIAL TEÓRICO	10
2.1	Desenvolvimento (<i>Dev</i>) e Operações(<i>Ops</i>)	10
2.2	O ciclo de vida <i>DevOps</i>	11
2.3	CI (Integração Contínua)	12
2.4	CD (Entrega Contínua)	13
2.5	Relação entre Integração Contínua e Entrega Contínua	14
2.6	Esteira de <i>CI/CD (Pipeline)</i>	15
2.7	Arquivos de Configuração de <i>Pipeline</i>	15
2.8	Ferramentas de <i>CI/CD</i>	16
2.8.1	GitHub Actions	16
2.8.2	GitLab CI/CD	18
2.8.3	Jenkins	20
2.9	Benefícios e Desafios das Práticas de <i>CI/CD</i>	22
2.9.1	Benefícios detalhados	22
2.9.2	Desafios associados	23
3	TRABALHOS RELACIONADOS	24
4	MATERIAIS	25
4.1	Tecnologias de Apoio ao Projeto	25
5	MÉTODOS	26
5.1	Análise Funcional Preliminar	26
5.2	Estrutura do Estudo	27
5.3	Procedimento de Implementação	27
5.4	Ambiente de Execução	28

5.5	Cr�terios de Avalia��o	28
5.6	Tratamento dos Dados	29
	REFER�NCIAS	30

1 INTRODUÇÃO

A automação de processos no desenvolvimento de software, através de ferramentas de integração e entrega contínua (CI/CD), é uma prática essencial no contexto de *DevOps*^{2,1} e desenvolvimento ágil, facilitando a entrega rápida e confiável de software. À medida que a complexidade dos projetos aumenta, ferramentas como GitHub Actions¹, GitLab CI/CD² e Jenkins³ surgem como soluções importantes para automatizar processos e garantir qualidade e eficiência no ciclo de desenvolvimento.

No contexto de *DevOps*, a automação desses processos é fundamental para garantir a integração eficaz entre desenvolvimento e operações, promovendo maior colaboração entre as equipes e reduzindo o tempo necessário para a entrega de novas funcionalidades. Além disso, o uso de *pipelines*^{2,6} de CI/CD permite estruturar e padronizar a execução de testes, *builds*⁴ e *deploys*⁵, garantindo um fluxo de trabalho mais confiável e eficiente (Amazon Web Services, 2024).

A CI consiste na prática de integrar alterações de código frequentemente em um repositório compartilhado, onde são automaticamente verificadas e testadas. Essa abordagem minimiza riscos ao detectar erros precocemente, garantindo maior estabilidade ao desenvolvimento por meio de validações constantes a cada *commit*⁶. Dessa forma, eventuais falhas podem ser corrigidas rapidamente, evitando que problemas se propaguem para versões posteriores do *software* (FOWLER, 2024).

Já a CD assegura que o *software* esteja sempre em um estado pronto para implantação, permitindo lançamentos frequentes e previsíveis. Essa prática possibilita que as equipes otimizem continuamente seus produtos, identificando e resolvendo conflitos entre o código novo e o existente de maneira ágil. Com isso, a correção de erros torna-se mais rápida e frequente, reduzindo o tempo necessário para entregar novas funcionalidades e melhorando a experiência dos usuários (RedHat, 2025).

A seleção das ferramentas Jenkins, GitLab CI/CD e GitHub Actions para este estudo é fundamentada em sua ampla adoção e relevância no cenário atual de desenvolvimento de software. Essas ferramentas são reconhecidas não apenas por sua popularidade e uso em projetos de diversos portes, mas também por possuírem documentação extensa e detalhada, o que facilita a implementação e a resolução de problemas. Além disso, todas oferecem planos

¹ <https://github.com/>

² <https://about.gitlab.com/>

³ <https://www.jenkins.io/>

⁴ Build (construção) é o processo de transformar código-fonte em um formato executável. Isso geralmente envolve compilação ou empacotamento de arquivos.

⁵ Deploy (implantação) é o processo de disponibilizar uma aplicação ou sistema em um ambiente de execução (como produção, homologação ou desenvolvimento) para que possa ser utilizado pelos usuários ou teste.

⁶ Um commit representa uma unidade de alteração no código-fonte registrada no histórico de um repositório, geralmente acompanhada de uma mensagem descritiva.

gratuitos, tornando-se acessíveis para equipes que buscam iniciar ou aprimorar suas práticas de CI/CD sem custos iniciais. Segundo relatórios e pesquisas de mercado, como o JetBrains Developer Ecosystem Survey (2023), essas três ferramentas estão entre as mais utilizadas para automação de pipelines de integração e entrega contínua. O Jenkins, uma ferramenta de código aberto consolidada, é reconhecido por sua flexibilidade e extenso ecossistema de plugins (Jenkins, 2024a). O GitLab CI/CD, por sua vez, se destaca por oferecer uma plataforma integrada de DevOps, unindo gestão de código, (CI/CD) e monitoramento em um único ambiente (GitLab, Inc., 2024). Já o GitHub Actions, com sua integração nativa ao GitHub, permite a automação de fluxos de trabalho diretamente nos repositórios, facilitando a adoção por equipes que já utilizam a plataforma (GitHub, Inc., 2024a). Além disso, oferece diversos *templates*⁷ que simplificam a criação de *pipelines* (GitHub, Inc., 2025).

1.1 Considerações iniciais

CI e CD são práticas fundamentais no desenvolvimento de *software* moderno, especialmente em ambientes que adotam metodologias ágeis e *DevOps*. Essas práticas são implementadas por meio de *pipelines* que automatizam a execução das etapas de compilação, testes, integração, revisão e implantação, permitindo um ciclo de desenvolvimento acelerado e com maior confiabilidade no processo de entrega de software (FOWLER, 2024). De acordo com Humble e Farley (2010), CI e CD são essenciais para reduzir o tempo entre o desenvolvimento de uma funcionalidade e sua disponibilização, além de mitigar os riscos de falhas em produção.

Cada uma dessas plataformas apresenta suas características particulares, vantagens e desafios que podem impactar diretamente a eficiência de equipes de desenvolvimento. A escolha de uma ferramenta adequada pode afetar desde a facilidade de implementação das esteiras até a eficiência geral da equipe e do sistema (GitLab, Inc., 2025c). Como afirmado por Duvall Paul M. e Glover (2007), o sucesso na adoção de CI/CD depende não apenas da escolha das ferramentas, mas também da integração dessas com as práticas da equipe e do projeto.

GitHub Actions e GitLab oferecem integração nativa com o controle de versão, o que proporciona uma experiência mais fluida para desenvolvedores que já utilizam essas ferramentas. Por exemplo, o GitHub Actions, integrado ao ecossistema do GitHub, permite automação de *workflows*⁸ diretamente no repositório com *templates* prontos e integrações de ferramentas de terceiros pelo GitHub Marketplace⁹ (GitHub, Inc., 2024b), (GitLab, Inc., 2024).

Jenkins, por outro lado, como uma ferramenta open-source altamente configurável, permite uma flexibilidade maior em termos de customização e uso de plugins (Jenkins, 2024a). Entretanto, essa flexibilidade vem acompanhada de uma complexidade maior de configuração e

⁷ <https://docs.gitlab.com/development/cicd/templates/>

⁸ Workflow ou fluxo de trabalho em desenvolvimento de software e CI/CD refere-se a uma sequência definida de etapas ou tarefas automatizadas que são executadas em uma ordem específica para atingir um objetivo, como compilar, testar e implantar código.

⁹ <https://github.com/marketplace>

uma curva de aprendizado mais acentuada. Segundo Riungu-Kalliosaari, Mäkinen e Tyrväinen (2016), ferramentas como Jenkins, apesar de seu potencial de flexibilidade, podem ser mais adequadas para equipes com maior maturidade técnica, sendo preferidas em cenários que exigem customizações específicas e fluxos de trabalho complexos.

Neste cenário, existe uma demanda crescente por estudos que avaliem de forma comparativa essas plataformas de CI/CD, fornecendo diretrizes claras para diferentes contextos de uso (SHAHIN; BABAR; ZHU, 2017). Este trabalho insere-se nesse contexto ao realizar uma análise comparativa, considerando aspectos como facilidade de uso, complexidade de configuração e desempenho de execução dos pipelines.

1.2 Objetivos

Os objetivos deste trabalho consistem em realizar uma análise comparativa entre as ferramentas de CI/CD: Jenkins, GitLab CI/CD e GitHub Actions. Os resultados alcançados visam auxiliar a comunidade acadêmica e equipes de desenvolvimento de software na escolha da ferramenta mais adequada para seus projetos.

1.2.1 Objetivo geral

Avaliar e comparar as funcionalidades relacionadas à CI/CD das ferramentas: GitHub Actions, GitLab CI/CD e Jenkins.

1.2.2 Objetivos específicos

- Identificar as principais características e limitações de cada ferramenta, comparando suas funcionalidades;
- Oferecer recomendações práticas para a escolha da ferramenta mais adequada a diferentes contextos de desenvolvimento, baseadas nos resultados obtidos;
- Avaliar a facilidade de configuração, utilizando um conjunto padronizado de tarefas, para comparar o tempo envolvido na criação das *pipelines*.

1.3 Justificativa

A justificativa para este trabalho reside na importância de coletar, analisar e fornecer informações comparativas das funcionalidades das ferramentas: Jenkins, GitLab CI/CD e GitHub Actions. Tal abordagem visa, eventualmente, auxiliar equipes de desenvolvimento de software no processo de escolha da solução mais adequada às suas necessidades. Diante da crescente complexidade dos projetos de *software* modernos e da fundamentalidade das práticas de CI/CD

para a automação de processos e o aumento da eficiência no ciclo de desenvolvimento de software (FOWLER, 2024). Este estudo, portanto, busca contribuir para a otimização dos fluxos de trabalho e a melhoria contínua na entrega de software, preenchendo uma lacuna na pesquisa sobre qual ferramenta melhor se adapta a diferentes contextos.

A escolha das plataformas GitHub, GitLab e Jenkins reflete a busca por soluções amplamente adotadas no mercado e bem documentadas, que atendem às necessidades de equipes com diferentes perfis e contextos e que oferecem planos gratuitos, segundo a pesquisa (blue-light.co, 2025).

O estudo também pode contribuir para a literatura acadêmica e prática, auxiliando equipes de desenvolvimento e engenheiros de *software* na escolha da ferramenta mais apropriada para suas necessidades específicas, contribuição importante para o mercado de trabalho, onde a adoção de práticas CI/CD é cada vez mais indispensável para a manutenção da qualidade do *software* e do ciclo de entrega (RIUNGU-KALLIOSAARI; MÄKINEN; TYRVÄINEN, 2016).

2 REFERENCIAL TEÓRICO

O referencial teórico deste trabalho se concentra em conceitos e práticas relacionadas a *DevOps*, mais especificamente CI e CD, que desempenham um papel relevante no desenvolvimento de *software*. Para contextualizar o trabalho, este capítulo será dividido em seções que abordam as definições de CI e CD, as *pipelines* de CI/CD e as ferramentas em foco (GitHub Actions, GitLab CI/CD e Jenkins), além de uma análise das práticas e benefícios associados a cada uma delas.

2.1 Desenvolvimento (*Dev*) e Operações(*Ops*)

DevOps é uma abordagem cultural, filosófica e técnica que busca integrar as equipes de desenvolvimento e operações, promovendo colaboração, automação e monitoramento contínuo em todas as etapas do desenvolvimento de *software*, desde a codificação até a produção (Amazon Web Services, 2024).

De acordo com Bass, Weber e Zhu (2020), essa abordagem visa melhorar a agilidade das organizações por meio da integração dos processos de desenvolvimento e operação, reduzindo o tempo entre a implementação de uma alteração e sua disponibilização ao usuário final. Além disso, essa prática está associada à melhoria da qualidade do *software*, maior frequência de entrega e maior estabilidade nos ambientes de produção.

Segundo Riungu-Kalliosaari, Mäkinen e Tyrväinen (2016), os benefícios da adoção de *DevOps* incluem maior eficiência no ciclo de vida do desenvolvimento, *feedback* mais rápido, melhor colaboração entre equipes e maior capacidade de resposta a mudanças. No entanto, os autores também destacam desafios, como a necessidade de mudança cultural, capacitação das equipes e adaptação de ferramentas e processos.

O *DevOps* serve como base para práticas como integração contínua e entrega contínua, permitindo que essas sejam implementadas de forma eficaz e integrada aos fluxos de trabalho modernos (RedHat, 2025).

Historicamente, muitas organizações seguiram modelos tradicionais nos quais as equipes de desenvolvimento, testes e operações atuavam de forma isolada, com responsabilidades bem definidas e pouco diálogo entre os setores. Essa separação gerava diversos desafios, como a ausência de uma linguagem comum, dificuldade na troca de conhecimento e barreiras culturais que impactavam diretamente na eficiência e qualidade do *software* entregue (Amazon Web Services, 2024).

A abordagem *DevOps* surge como uma resposta a esse modelo fragmentado, propondo a quebra desses silos organizacionais e incentivando uma mentalidade colaborativa. Ao integrar desenvolvimento, testes e operações em um fluxo contínuo, é possível alinhar os objetivos das equipes e promover uma visão mais unificada do produto e do cliente.

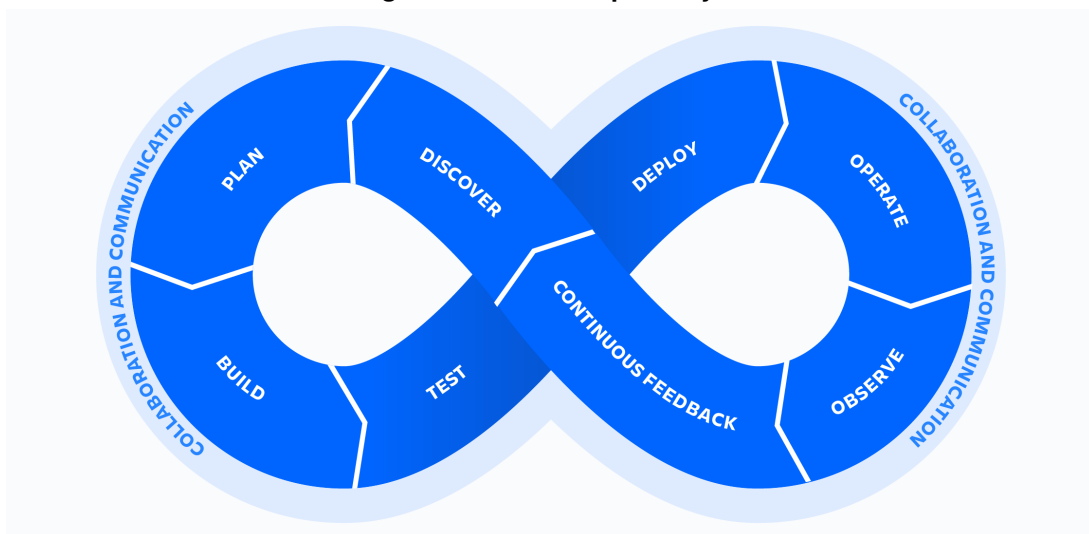
Com isso, práticas como automação de testes, entrega contínua e monitoramento em tempo real tornam-se viáveis e eficazes, reduzindo o tempo de entrega, aumentando a qualidade do *software* e permitindo respostas mais ágeis a mudanças e incidentes. Além disso, a criação de uma linguagem compartilhada e a busca por objetivos comuns ajudam a fortalecer a cultura organizacional e a disseminação de conhecimento entre os times.

Essa transformação, no entanto, exige mais do que apenas a adoção de ferramentas: requer uma mudança cultural profunda, onde valores como confiança, transparência e responsabilidade compartilhada passam a ser fundamentais no dia a dia das equipes.

2.2 O ciclo de vida *DevOps*

O ciclo *DevOps* é composto por diversas etapas que representam processos, ferramentas e práticas específicas. Quando integradas, essas fases visam acelerar o desenvolvimento de *software*, desde a concepção inicial até a entrega ao usuário final.

Figura 1 – The DevOps lifecycle



Fonte: (Atlassian, 2025).

A Figura 1 ilustra o ciclo de vida *DevOps*, no qual as fases estão organizadas em um fluxo contínuo e iterativo. Conforme descrito pela Atlassian¹ cada etapa desempenha um papel fundamental para viabilizar entregas ágeis e confiáveis, com ênfase na automação e na melhoria contínua (Atlassian, 2025). Essas fases estão interconectadas de forma cíclica, permitindo a evolução constante e a automatização progressiva dos processos.

A seguir, detalhamos cada uma das fases representadas na figura:

1. **Descoberta (*Discover*):** Antes do desenvolvimento, ocorre a análise e identificação de oportunidades e riscos, permitindo que as equipes entendam melhor o contexto do projeto e tomem decisões informadas.

¹ A Atlassian é uma empresa de *software* conhecida por ferramentas como Jira, Confluence e Bitbucket.

2. **Planejamento (*Plan*):** Corresponde ao momento em que as equipes definem as funcionalidades a serem implementadas, priorizando tarefas com base em metas estratégicas e necessidades do cliente.
3. **Desenvolvimento (*Build*):** Nesta fase, o código é desenvolvido com o auxílio de sistemas de controle de versão, como o *Git*². A colaboração entre desenvolvedores é facilitada, e práticas como integração contínua (*CI*) são aplicadas para garantir qualidade desde o início.
4. **Testes (*Test*):** O código desenvolvido passa por testes automatizados para assegurar a estabilidade e qualidade do sistema. Essa etapa permite detectar falhas de forma precoce, reduzindo custos de correção.
5. **Implantação (*Deploy*):** Utilizando práticas de entrega contínua (*CD*), as equipes automatizam a publicação de novas versões do *software*. Para reduzir falhas e interrupções, são utilizadas técnicas de deploy gradual, que promovem uma transição mais segura entre versões e contribuem para a estabilidade do sistema.
6. **Operações (*Operate*):** Esta fase envolve o gerenciamento da infraestrutura e dos serviços em produção, garantindo disponibilidade, desempenho e resiliência dos sistemas.
7. **Monitoramento (*Observe*):** Ferramentas de monitoramento e análise de logs são utilizadas para acompanhar o funcionamento do sistema, identificar anomalias e embasar melhorias futuras.
8. **Feedback Contínuo (*Continuous Feedback*):** Após a entrega, feedbacks de usuários e métricas de operação são analisados para retroalimentar o ciclo de desenvolvimento, gerando ajustes e novas melhorias.

2.3 CI (Integração Contínua)

CI é uma prática de desenvolvimento de *software* que incentiva os desenvolvedores a integrar suas mudanças de código frequentemente em um repositório³. Esse processo é geralmente automatizado para compilar, testar e verificar o código, ajudando a identificar e resolver erros rapidamente, sendo essencial para evitar problemas de integração e reduzir retrabalho, promovendo um desenvolvimento mais rápido e confiável.

² Git é um sistema de controle de versão distribuído amplamente utilizado para rastrear alterações no código-fonte durante o desenvolvimento de *software*.

³ Repositório é o local central onde o código-fonte de um projeto é armazenado e gerenciado, geralmente com suporte a controle de versão como *Git*.

Verificar automaticamente cada *commit* reduz o risco de falhas acumuladas e contribui para a melhoria da qualidade do *software*. Para garantir a integridade das *branches*⁴ principais, utiliza-se *pipelines* com regras que exigem a aprovação em todos os testes unitários e de integração antes que qualquer alteração seja incorporada. Essa abordagem evita que código instável seja introduzido, assegurando a qualidade e a estabilidade do *software*. Segundo Fowler (2024), práticas como essas são especialmente úteis em ambientes que adotam metodologias ágeis, pois facilitam a manutenção de versões consistentes e reduzem a introdução de falhas em produção.

Além de aumentar a velocidade e a frequência das integrações, também contribui para um processo mais seguro e previsível, fornecendo um *feedback* imediato sobre o impacto das mudanças realizadas. De acordo com Duvall Paul M. e Glover (2007) essa prática permite, através dos testes automatizados integrados na *pipeline*, que equipes de desenvolvimento identifiquem problemas de forma incremental e continuada, resultando em um desenvolvimento menos propenso a falhas e com entregas mais consistentes.

2.4 CD (Entrega Contínua)

CD é uma prática de desenvolvimento de *software* que automatiza a implantação de novas versões, garantindo que o *software* esteja sempre pronto para ser atualizado de forma rápida e segura. Sucede a CI e assegura que cada versão aprovada nos testes automatizados possa ser implantada imediatamente em ambientes de produção ou homologação, minimizando a intervenção manual e reduzindo os riscos de falhas.

De acordo com a Amazon Web Services (2024)⁵ a entrega contínua está alinhada com os princípios *DevOps*, promovendo colaboração entre as equipes de desenvolvimento e operações para tornar o processo de implantação mais ágil e seguro. A automação reduz falhas humanas e aumenta a consistência das entregas, o que é fundamental para empresas que adotam práticas ágeis e ciclos frequentes de entrega.

Atlassian (2025) destaca que a *CD* não só reduz o tempo entre a conclusão do desenvolvimento e a entrega de novos recursos, mas também melhora a velocidade na identificação e correção de erros. Isso permite que as equipes implementem rapidamente atualizações e correções, mantendo um ciclo de desenvolvimento contínuo e altamente responsivo.

A entrega contínua se consolida como um pilar essencial para *pipelines* modernas; sua adoção permite que equipes respondam com rapidez a demandas técnicas e de negócios, mantendo a qualidade e a confiabilidade das entregas. Dessa forma, a prática não apenas otimiza

⁴ *Branches* são ramificações independentes dentro de um repositório, utilizadas para desenvolvimento paralelo e controle de versões.

⁵ A *Amazon Web Services (AWS)* é uma plataforma de serviços de computação em nuvem oferecida pela amplamente utilizada para desenvolvimento, hospedagem, automação e escalabilidade de aplicações.

o fluxo de trabalho, mas também estabelece as bases para estratégias mais avançadas de implantação (HUMBLE; FARLEY, 2010).

2.5 Relação entre Integração Contínua e Entrega Contínua

CI é um componente essencial para CD, pois prepara o *software* para a entrega contínua, certificando que ele seja frequentemente integrado e testado. Enquanto CI foca na verificação constante de cada alteração de código, CD expande essa prática ao automatizar a disponibilização do *software* em diferentes ambientes, como desenvolvimento, homologação e produção, garantindo que ele esteja tecnicamente apto para ser implantado a qualquer momento (RedHat, 2025). Essa relação sinérgica assegura um fluxo de desenvolvimento contínuo e altamente responsivo.

É possível visualizar a relação intrínseca entre CI e CD na Figura 1, que ilustra um ciclo contínuo e iterativo de desenvolvimento e entrega de software. Este processo pode ser interpretado em fases interligadas:

A porção esquerda do diagrama representa a CI, abrangendo os primeiros estágios do ciclo de desenvolvimento. O processo se inicia com o Planejamento (1), onde são definidos os requisitos, funcionalidades e objetivos do projeto. Em seguida, o código é Escrito (2) com base nesse planejamento. As alterações realizadas são então Compiladas (3) em uma versão executável e submetidas a Testes (4) automatizados. Esses testes garantem que as mudanças não introduzam falhas e que o código continue integrado e funcional. A CI, portanto, atua como um filtro de qualidade contínuo, validando o software a cada pequena alteração. Essa validação contínua é fundamental para que a entrega contínua possa ocorrer com segurança.

A porção direita do diagrama representa a CD, que depende diretamente do sucesso da CI para automatizar a entrega e a operação do software. A CD só pode atuar com eficiência se tiver como base um software que foi corretamente integrado e testado previamente. Após a conclusão dos testes da CI, o software é empacotado para Lançamento (5), e então Implantado (6) nos ambientes de destino. As etapas seguintes incluem a Operação (7), onde o sistema é mantido ativo e monitorado, e o Monitoramento (8), que gera dados sobre o desempenho e uso da aplicação. Esses dados alimentam o ciclo de Feedback Contínuo, promovendo a melhoria contínua. Em suma, a CD depende da confiabilidade e estabilidade proporcionadas pela CI para garantir que as implantações sejam frequentes, automáticas e seguras.

A forma do diagrama enfatiza a natureza cíclica e contínua do DevOps, onde CI e CD são elementos cruciais interligados. A colaboração e comunicação permeiam todo o ciclo, garantindo que as equipes trabalhem juntas de forma eficaz, enquanto o feedback contínuo permite a melhoria contínua do processo e do produto.

2.6 Esteira de CI/CD (Pipeline)

Uma esteira de CI/CD, ou *pipeline*, é uma série de etapas automatizadas que uma aplicação percorre desde a submissão do código até a implantação final em produção. Essa *pipeline* é responsável por automatizar processos essenciais, como compilação, execução de testes, integração, revisão e implantação, formando um fluxo contínuo que acelera e padroniza o ciclo de desenvolvimento de *software* (SHAHIN; BABAR; ZHU, 2017).

A criação e configuração das *pipelines* de CI/CD geralmente são feitas por meio de arquivos de configuração escritos em formatos como *YAML*⁶, que permitem descrever as etapas e os comandos necessários em uma estrutura organizada e legível. Esses arquivos especificam os passos que o código deve seguir no processo de integração e entrega contínua, como executar testes unitários, compilar a aplicação, realizar verificações de segurança e, finalmente, implantar a aplicação em um ambiente de desenvolvimento, homologação e produção (GitLab, Inc., 2024).

Para realizar essas etapas, a configuração das *pipelines* muitas vezes incorpora comandos e scripts em Shell Script⁷, que são amplamente usados para manipular arquivos, configurar variáveis de ambiente, automatizar processos e controlar o comportamento das etapas no *pipeline* (GitLab, Inc., 2024). Esse nível de configuração detalhado permite que as *pipelines* sejam altamente personalizáveis e adequadas às necessidades específicas de cada projeto, facilitando a adaptação das equipes a requisitos distintos e acelerando o ciclo de desenvolvimento (BASS; WEBER; ZHU, 2020).

As *Pipelines CI/CD* desempenham um papel crucial na organização e automação de todo o processo de entrega de *software*, não apenas reduzem a intervenção manual, mas também promovem a consistência entre builds e implantações, minimizando erros e aumentando a confiança na entrega do *software*. Essas *pipelines* desempenham um papel central na metodologia *DevOps*, uma vez que permitem a integração entre desenvolvimento e operações, melhorando a colaboração entre equipes e permitindo entregas rápidas e frequentes (HUMBLE; FARLEY, 2010).

2.7 Arquivos de Configuração de Pipeline

Os arquivos de configuração de *pipeline* são documentos estruturados que definem o fluxo de execução automatizado de processos de integração e entrega contínua. Esses arquivos especificam tarefas como compilação, testes, deploy e outras etapas essenciais no desenvolvi-

⁶ *YAML* (Yet Another Markup Language) é uma linguagem de serialização de dados legível por humanos, amplamente utilizada para configuração de sistemas e definição de fluxos.

⁷ *Shell Script* é um conjunto de comandos escritos em um interpretador de linha de comando (como *Bash*), utilizado para automatizar tarefas em sistemas operacionais Unix-like, como manipulação de arquivos, execução de programas e controle de fluxo.

mento de *software*. Através desses arquivos, as ferramentas de *CI/CD* organizam e coordenam a execução automatizada do *pipeline*.

A maioria dessas ferramentas utiliza *YAML* como formato de escrita para os arquivos de configuração. O *YAML* é uma linguagem de serialização de dados que se destaca pela simplicidade, legibilidade e estrutura hierárquica baseada em indentação. Isso facilita a escrita e manutenção de *pipelines*, permitindo que desenvolvedores descrevam de forma declarativa as etapas do fluxo de trabalho (Red Hat, 2023).

Conforme Bass, Weber e Zhu (2020), a automação de *pipelines* reduz erros humanos e melhora a eficiência dos processos de desenvolvimento. Além disso, Humble e Farley (2010) destacam que a configuração declarativa, como a oferecida pelo *YAML*, promove reprodutibilidade e previsibilidade nas execuções, garantindo que os builds e deploys ocorram de maneira padronizada.

Dessa forma, a adoção de arquivos de configuração de *pipeline* escritos em *YAML* tornou-se uma prática essencial em *DevOps* e desenvolvimento moderno, possibilitando maior controle, automação e escalabilidade nos projetos.

Exemplos práticos de arquivos de configuração em *YAML* podem ser encontrados em seções posteriores deste trabalho, contemplando as ferramentas analisadas — GitHub Actions, GitLab *CI/CD* e Jenkins. Nessas seções, são apresentados trechos reais de códigos utilizados para configuração de *pipelines*.

2.8 Ferramentas de *CI/CD*

2.8.1 GitHub Actions

O GitHub Actions é uma solução de automação integrada ao GitHub, lançada em 2018. Ele permite que desenvolvedores configurem *pipelines* de *CI/CD* utilizando arquivos *YAML* diretamente no repositório do projeto. Seu principal diferencial é a integração nativa com o GitHub, simplificando a configuração de fluxos de trabalho (GitHub, Inc., 2024a).

Com o GitHub Actions, é possível definir *triggers* (eventos) para disparar ações automáticas, como testes unitários, *build* (compilação) da aplicação e *deploy* (implantação). O fluxo de trabalho é estruturado em *jobs* (trabalhos), que contêm *steps* (etapas) específicas para execução (GitHub, Inc., 2024a).

Uma das vantagens do GitHub Actions é a sua vasta biblioteca de ações reutilizáveis disponibilizadas no GitHub Marketplace, permitindo que os usuários aproveitem configurações pré-definidas para automação de tarefas sem necessidade de escrever *scripts* do zero.

No entanto, o GitHub Actions pode apresentar custos adicionais dependendo do uso de runners hospedados pelo próprio GitHub, o que pode ser um fator limitante para projetos

maiores. Além disso, sua configuração, embora simplificada, pode ser menos flexível do que ferramentas como o Jenkins.

Listagem 1 – Exemplo arquivo de configuração de *pipeline* do GitHub

```

1 name: Exemplo Trigger e Jobs
2
3 on: # Trigger: Descreve quando executar a esteira
4   push:
5     branches:
6       - main
7
8 jobs: # Jobs: Descreve as ações que a esteira executará
9   build-and-test:
10     runs-on: ubuntu-latest
11
12     steps:
13     - name: Checkout code
14       uses: actions/checkout@v3
15
16     - name: Compilar código
17       run: npm run build
18
19     - name: Testar código
20       run: npm run test

```

Fonte: Autoria própria (2025).

A Figura 1 apresenta um exemplo de *workflow* definido em um arquivo *YAML* para automação com GitHub Actions. Esse fluxo de trabalho é acionado automaticamente sempre que ocorre um *push*⁸ na branch *main* do repositório.

O *workflow* consiste em um único *job*, chamado "*build-and-test*", que é executado no ambiente *Ubuntu* na versão mais recente disponível. Esse *job* é estruturado em três etapas principais, cada uma desempenhando uma função específica no processo de automação:

1. *Checkout* do código-fonte – Responsável por clonar o repositório para o ambiente de execução.
2. Compilação do código – Utiliza o comando *npm run build* para processar a construção da aplicação.
3. Execução de testes – Roda a suíte de testes automatizados com *npm run test*, garantindo que o código está funcionando corretamente.

Como demonstrado na Figura 1, o GitHub Actions permite configurar *triggers* (eventos acionadores) que determinam quando um *workflow* será executado. No exemplo, a execução

⁸ Um *push* é uma ação realizada no Git que envia as alterações no repositório local para o repositório remoto, atualizando-o com os novos *commits*.

ocorre sempre que um *push* é feito na *branch main*, mas é possível definir outros eventos, como *pull requests*⁹ ou execuções manuais, conforme a necessidade do projeto.

Essa flexibilidade é essencial para otimizar o uso de recursos e garantir que a automação aconteça apenas quando necessário. Além disso, a configuração manual pode ser útil para cenários onde é exigida uma aprovação humana, como um *deploy* em produção, assegurando um maior controle sobre o processo.

2.8.2 GitLab CI/CD

O GitLab CI/CD é a solução de automação nativa do GitLab, lançada em 2015, que permite a criação e execução de *pipelines* diretamente dentro da plataforma, utilizando arquivos *YAML* armazenados no repositório. O GitLab CI/CD possibilita a definição de fluxos automatizados para tarefas como compilação, testes e implementação, garantindo maior eficiência no desenvolvimento de *software* (GitLab, Inc., 2024).

Um dos grandes diferenciais do GitLab CI/CD é sua abordagem baseada em *stages* (estágios), onde cada *job* (trabalho) executa uma parte do *pipeline* de forma organizada e sequencial (GitLab, Inc., 2025a). Além disso, a ferramenta oferece suporte a *runners* dedicados, permitindo que os *pipelines* sejam executados em servidores próprios ou na infraestrutura compartilhada do GitLab. Essa flexibilidade torna o GitLab CI/CD altamente escalável para projetos de diferentes tamanhos (GitLab, Inc., 2025b).

O ponto forte do GitLab CI/CD é a sua integração com o próprio GitLab, facilitando a configuração de *pipelines* sem necessidade de ferramentas externas. A plataforma também disponibiliza *templates* pré-configurados para diversas linguagens e *frameworks*, agilizando a implementação de *pipelines* personalizados para diferentes tipos de projetos.

⁹ *Pull request* é uma solicitação para que alterações propostas em um repositório sejam revisadas e possivelmente incorporadas à *branch principal*, comum em repositórios baseados em Git.

Listagem 2 – Exemplo arquivo de configuração de *pipeline* do GitLab

```
1 workflow :
2   rules :
3     - if: '$CI_COMMIT_BRANCH == "main" '
4
5 build-job :
6   stage: build
7   image: node:18
8   script:
9     - npm install
10    - npm run build
11  artifacts :
12    paths:
13      - dist/
14
15 test-job :
16   stage: test
17   image: node:18
18   script:
19     - npm test
```

Fonte: Autoria própria (2025).

A Figura 2 ilustra um exemplo de *pipeline* no GitLab CI/CD, configurado para ser acionado automaticamente sempre que um push é feito na branch main. Esse *pipeline* segue uma estrutura modular, garantindo organização, reutilização e escalabilidade no processo de integração e entrega contínua.

1. Build da aplicação – O job build-job instala as dependências e executa npm run build, gerando a versão compilada do projeto.
2. Execução de testes – O job test-job executa npm test, validando a integridade do código antes de um possível deploy.

Cada job pertence a um stage específico e só será executado quando todos os jobs do stage anterior forem concluídos com sucesso.

O job build-job pertence ao stage build e será responsável por compilar a aplicação. Ele utiliza a imagem oficial do Node.js (versão 18) como ambiente de execução; após executar a compilação, armazena os arquivos gerados durante a execução do job para serem utilizados em estágios posteriores. Neste caso, a pasta dist/ (que contém os arquivos da build) será salva e disponibilizada para o próximo estágio.

Essa esteira modular do GitLab CI/CD permite uma organização eficiente das etapas de automação, garantindo que a aplicação seja construída e testada de forma controlada. Essa abordagem modular e declarativa facilita a manutenção e escalabilidade dos *pipelines*, permitindo que equipes de desenvolvimento automatizem seus fluxos de trabalho de maneira eficaz (FOWLER, 2024).

2.8.3 Jenkins

O Jenkins é uma ferramenta de automação *open-source* amplamente utilizada para integração e entrega contínua. Criado originalmente como parte de outro projeto, se tornou independente em 2011, se destaca por sua flexibilidade e extensibilidade, permitindo que equipes configurem *pipelines* altamente personalizados para atender a diferentes cenários de desenvolvimento (Jenkins, 2024a).

Diferente de soluções como GitHub Actions e GitLab CI/CD, que possuem uma integração nativa com suas respectivas plataformas, o Jenkins opera de forma autônoma, sendo possível utilizá-lo com qualquer provedor de repositório Git ou outra ferramenta de versionamento. Sua configuração pode ser feita tanto de forma declarativa utilizando arquivos *YAML* ou *Groovy* quanto de maneira imperativa, por meio da interface gráfica (Jenkins, 2024d).

Uma das maiores vantagens do Jenkins é a sua extensibilidade por meio de *plugins*, com milhares de opções disponíveis atualmente, possibilitando integração com diversas tecnologias, como *Docker*, *Kubernetes*, bancos de dados, ferramentas de monitoramento e provedores de nuvem (Jenkins, 2024b). Esse ecossistema expansível torna o Jenkins altamente adaptável a diferentes tipos de projetos.

Além disso, o Jenkins permite a execução de *pipelines* em agentes distribuídos, conhecidos como *nodes*, possibilitando a escalabilidade horizontal da infraestrutura de CI/CD (Jenkins, 2024c). Essa abordagem é especialmente útil para grandes equipes e projetos complexos, onde múltiplos *pipelines* podem ser executados simultaneamente, reduzindo o tempo de build e testes.

Outro diferencial do Jenkins é a possibilidade de configurar *pipelines* a partir de *templates* pré-definidos, por meio do *Jenkins Shared Libraries*. Com isso, empresas podem padronizar seus *pipelines* e compartilhar lógica reutilizável entre diferentes projetos, promovendo consistência e eficiência na automação (Jenkins, 2025a).

Listagem 3 – Exemplo arquivo de configuração de *pipeline* do Jenkins usando Groove

```

1 pipeline {
2   agent any
3   stages {
4     stage('Build') {
5       steps {
6         checkout scm
7         sh 'npm install'
8         sh 'npm run build'
9       }
10      post {
11        success {
12          archiveArtifacts artifacts: 'dist/**', fingerprint: true
13        }
14      }
15    }
16    stage('Test') {
17      steps {
18        sh 'npm test'
19      }
20    }
21  }
22 }

```

Fonte: Autoria própria (2025).

Esse *pipeline* também segue uma abordagem modular, onde os estágios são organizados sequencialmente para garantir um fluxo estruturado de *build* e testes, assim como nas figuras 1 e 2.

A estrutura do *pipeline* no Jenkins permite um alto nível de personalização. Os agentes (definidos pela diretiva *agent*) são responsáveis por executar as tarefas do *pipeline*. No exemplo acima, o agente é configurado como *any*, indicando que o *pipeline* pode rodar em qualquer ambiente disponível. No entanto, é possível definir agentes específicos, como máquinas dedicadas ou contêineres *Docker* (Jenkins, 2024c).

Além disso, o uso de *archiveArtifacts* permite armazenar os arquivos gerados no estágio de *build* para serem reutilizados posteriormente (Jenkins, 2025b). Isso é útil para *pipelines* que envolvem múltiplas fases, como testes em ambientes distintos e *deploy* em produção.

Com sua abordagem flexível, escalável e altamente extensível, o Jenkins continua sendo uma das ferramentas mais robustas para *CI/CD*. Ele permite integração com praticamente qualquer ferramenta do ecossistema *DevOps*, oferecendo um nível avançado de personalização para equipes que buscam controle total sobre seus *pipelines* (Jenkins, 2024a).

2.9 Benefícios e Desafios das Práticas de CI/CD

A adoção de práticas de CI/CD traz diversos benefícios, incluindo maior agilidade na entrega de *software*, redução de riscos e melhoria na qualidade do produto final (SHAHIN; BABAR; ZHU, 2017). No entanto, também apresenta desafios, como a necessidade de uma mudança cultural dentro das equipes de desenvolvimento e a complexidade na configuração inicial das ferramentas. Riungu-Kalliosaari, Mäkinen e Tyrväinen (2016) enfatizam a importância de entender esses desafios para garantir uma transição bem-sucedida para práticas de *DevOps*.

2.9.1 Benefícios detalhados

- **Entrega Rápida e Frequente:** A automação de compilação, testes e implantação permite que o software seja entregue aos usuários de forma mais rápida e com maior frequência. Isso acelera o ciclo de *feedback* e permite que novas funcionalidades e correções sejam disponibilizadas prontamente. A CD, por exemplo, garante que o *software* esteja sempre pronto para implantação, possibilitando lançamentos frequentes e previsíveis (Amazon Web Services, 2025a).

- **Melhoria da Qualidade do Software:** A integração frequente e os testes automatizados garantem que as alterações de código sejam continuamente verificadas e testadas. Isso minimiza riscos ao detectar erros precocemente, assegurando a estabilidade e a qualidade do software através de validações constantes a cada commit (Amazon Web Services, 2025b). A identificação e correção de erros tornam-se mais ágeis e frequentes.

- **Redução de Riscos:** Ao validar o código em pequenas partes e de forma contínua, as falhas são detectadas e corrigidas antes que se propaguem e se tornem mais complexas e caras de resolver (FOWLER, 2024). Isso reduz o risco de introduzir código instável em ambientes de produção. A automação também minimiza erros humanos inerentes a processos manuais de implantação.

- **Colaboração Aprimorada:** As práticas de CI/CD promovem uma maior colaboração entre as equipes de desenvolvimento e operações (*DevOps*), alinhando objetivos e fortalecendo a cultura organizacional. O feedback contínuo e a transparência dos processos incentivam a comunicação e a responsabilidade compartilhada (RedHat, 2025).

- **Consistência e Reprodutibilidade:** As pipelines de CI/CD padronizam a execução de todas as etapas do desenvolvimento, como compilação, testes e implantação. Isso garante que os builds e deploys ocorram de maneira padronizada, promovendo reprodutibilidade e previsibilidade nas execuções (HUMBLE; FARLEY, 2010).

2.9.2 Desafios associados

- **Mudança Cultural:** Um dos maiores desafios é a necessidade de uma transformação cultural profunda dentro das organizações. As equipes precisam abandonar silos tradicionais e adotar uma mentalidade de colaboração, confiança e responsabilidade compartilhada (SAMANTHA, 2018).

- **Manutenção Contínua:** As pipelines e a infraestrutura de CI/CD podem exigir manutenção contínua e atualização, o que pode ser um recurso intensivo. A complexidade aumenta com o tamanho do projeto e o número de integrações.

- **Segurança:** A automação de todo o ciclo de entrega de software exige uma atenção redobrada à segurança, descartando a ideia de que a segurança é uma fase distinta e, em vez disso, garantindo que seja uma parte inerente do ciclo de vida de entrega do software. É crucial integrar práticas de *DevSecOps*¹⁰ nas pipelines para que vulnerabilidades sejam detectadas e mitigadas precocemente, reduzindo riscos ao longo do desenvolvimento e da implantação (Palo Alto Networks, 2025).

- **Seleção da Ferramenta Adequada:** Diante da diversidade de ferramentas de CI/CD disponíveis no mercado, as equipes enfrentam um desafio significativo ao tomar decisões sobre qual solução adotar. A escolha da solução mais adequada depende de múltiplos fatores, como a facilidade de configuração, a escalabilidade necessária, o suporte à personalização e a integração com o ecossistema tecnológico já existente da equipe. Avaliar essas características e ponderar os pontos fortes e limitações de cada ferramenta requer um entendimento aprofundado e pode influenciar diretamente a eficiência do fluxo de trabalho e a otimização dos recursos do projeto (GitLab, Inc., 2025c).

¹⁰ *DevSecOps* é uma abordagem que integra segurança em todas as fases do ciclo de vida de desenvolvimento de software, desde o planejamento e codificação até a implantação e operação, visando automatizar e incorporar práticas de segurança de forma contínua.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta estudos acadêmicos e técnicos anteriores que realizaram análises comparativas ou avaliaram ferramentas de CI/CD. A discussão tem como objetivo destacar resultados e critérios considerados relevantes por diferentes autores sobre as ferramentas estudadas, contextualizando GitHub Actions, GitLab CI/CD e Jenkins.

Shahin, Babar e Zhu (2017) realizaram uma revisão sistemática sobre abordagens, ferramentas e desafios relacionados à CI. O estudo destaca que Jenkins e GitHub Actions são amplamente adotados devido à sua flexibilidade e à capacidade de integração com outros serviços. Complementarmente, Riungu-Kalliosaari, Mäkinen e Tyrväinen (2016) destacam que a adoção dessas ferramentas contribui significativamente para a redução de erros manuais e aprimoramento da colaboração entre as equipes.

Na análise comparativa feita por GRACIANO (2023) entre GitHub Actions e GitLab CI/CD, critérios como tempo de execução dos pipelines, facilidade de configuração e integração com outras ferramentas são abordados. Nesse estudo, GitHub Actions destaca-se pela integração fluida com o ecossistema GitHub e facilidade de uso, enquanto o GitLab CI/CD apresenta vantagens na flexibilidade da configuração e no suporte a runners dedicados, que permitem execução em infraestruturas próprias.

Segundo LIMA (2021), ao comparar especificamente as funcionalidades de Jenkins e GitLab CI/CD, Jenkins oferece maior flexibilidade devido à sua natureza independente e extensa disponibilidade de plugins. No entanto, ressalta que isso resulta em uma complexidade maior na configuração e manutenção. Já o GitLab CI/CD é descrito como oferecendo uma experiência mais simplificada e intuitiva, particularmente para equipes que já utilizam o GitLab para gestão de código.

Estudos clássicos como o de Duvall Paul M. e Glover (2007) reforçam a importância de considerar fatores como tempo de execução dos pipelines, facilidade de uso e suporte abrangente para testes automatizados ao selecionar uma ferramenta de CI/CD. Adicionalmente, o relatório bluelight.co (2025) classifica GitHub Actions, GitLab CI/CD e Jenkins como líderes de mercado, destacando os pontos fortes e limitações de cada um e enfatizando que a escolha depende do nível de controle desejado sobre a infraestrutura e das características do ecossistema de desenvolvimento utilizado.

Esses estudos fornecem uma base importante que justifica a relevância da análise comparativa proposta neste trabalho. Ao considerar as funcionalidades, limitações, facilidade de configuração e o tempo exigido para implementação das pipelines, os resultados deste estudo buscam contribuir com orientações práticas para equipes de desenvolvimento na escolha da solução mais adequada às suas necessidades específicas.

4 MATERIAIS

Para a realização deste trabalho, foram selecionadas três ferramentas de integração e entrega contínua: Github Actions^{2.8.1}, Gitlab CI/CD^{2.8.2} e Jenkins^{2.8.3}. Essas plataformas foram escolhidas por sua ampla adoção na indústria de software e pelas diferentes abordagens que oferecem para a automação de pipelines.

Além das ferramentas de CI/CD, foram utilizados outros materiais essenciais para garantir a criação de um ambiente de testes controlado e a execução das pipelines de maneira padronizada. A seguir, são descritos os principais componentes empregados neste estudo, com foco em suas funções e características mais relevantes para a análise comparativa.

4.1 Tecnologias de Apoio ao Projeto

- **Git:** Sistema de controle de versão utilizado para o gerenciamento do código-fonte do projeto. Permitiu rastrear alterações, colaborar com diferentes plataformas e acionar pipelines de CI/CD por meio de *pull requests*.
- **Projeto Base (.NET):** As pipelines foram aplicadas sobre um projeto desenvolvido com a tecnologia .NET, escolhida por sua ampla adoção no desenvolvimento de aplicações corporativas. O projeto inclui testes automatizados com o framework xUnit, garantindo a validação contínua da lógica da aplicação durante a execução das pipelines.
- **.NET CLI e NuGet:** A interface de linha de comando do .NET foi utilizada nas etapas de compilação, execução de testes e publicação. O NuGet atuou como gerenciador de pacotes para restaurar as dependências do projeto nas etapas iniciais da automação.
- **Shell Script:** Utilizado para automatizar tarefas recorrentes dentro do ambiente de execução das pipelines, como configurações de ambiente e chamadas de comandos auxiliares.
- **YAML:** Linguagem de marcação empregada na definição das pipelines nas três plataformas analisadas. Sua estrutura declarativa e sintaxe simplificada facilitaram a criação, manutenção e leitura dos fluxos de automação.
- **Docker e Docker Hub:** O Docker foi utilizado para a criação de imagens containerizadas do projeto. As imagens geradas foram publicadas automaticamente no Docker Hub, promovendo o versionamento e a distribuição eficiente das aplicações.
- **Trivy:** Ferramenta de análise de segurança utilizada para escanear as imagens Docker em busca de vulnerabilidades conhecidas. Sua integração às pipelines permitiu verificar possíveis falhas antes da publicação das imagens no repositório, elevando o nível de segurança no processo de entrega contínua.

5 MÉTODOS

Este capítulo descreve os procedimentos que serão adotados para a execução da análise comparativa entre as ferramentas de integração e entrega contínua: GitHub Actions, GitLab CI/CD e Jenkins. As etapas metodológicas serão estruturadas com o objetivo de garantir a padronização dos testes e a reprodutibilidade dos resultados em um ambiente controlado. Inicialmente, será realizada uma análise funcional preliminar das ferramentas, com o intuito de comparar seus recursos nativos e capacidades de integração, antes da execução prática das pipelines.

5.1 Análise Funcional Preliminar

Antes da execução prática dos experimentos com as pipelines, será realizada uma análise funcional preliminar das ferramentas selecionadas. Essa análise terá como objetivo identificar e comparar os principais recursos nativos de cada plataforma de CI/CD, com foco em aspectos que impactam diretamente sua adoção e flexibilidade.

Os critérios considerados nessa etapa incluirão:

- **Integração com repositórios Git externos:** Verificação da compatibilidade com provedores Git como GitHub, GitLab e Bitbucket.
- **Suporte a ferramentas externas dentro da pipeline:** Capacidade de integrar ações ou *scripts* de terceiros, como ferramentas de análise de código, notificações, ou escaneamento de segurança.
- **Suporte a *runners* externos:** Verificação da possibilidade de configurar agentes de execução personalizados para as pipelines.
- **Disponibilidade em modelo *SaaS (Software as a Service)*:** Identificação de quais plataformas oferecem execução totalmente hospedada na nuvem ou que exigem instalação local.
- **Suporte a execução *self-hosted*:** Verificação da possibilidade de executar a ferramenta em ambientes locais, por meio de instalação em servidores próprios ou *containers*, permitindo maior controle sobre a infraestrutura e personalização do ambiente de execução.
- **Complexidade da configuração inicial:** Avaliação da quantidade de etapas necessárias para configurar a primeira *pipeline* funcional.
- **Suporte a múltiplos ambientes:** Verificação da capacidade da ferramenta de gerenciar *pipelines* com múltiplos estágios (ex.: desenvolvimento, homologação, produção).

- **Marketplace ou biblioteca de templates/actions:** Disponibilidade de um repositório público de exemplos, templates ou ações reutilizáveis para acelerar a construção das *pipelines*.
- **Interface de usuário (UI) para visualização da pipeline:** Existência e qualidade da interface gráfica para monitorar, depurar e controlar a execução das etapas da *pipeline*.
- **Suporte à paralelização de jobs:** Capacidade de executar múltiplos *jobs* simultaneamente, otimizando o tempo total de execução das *pipelines*.
- **Suporte a cache de dependências:** Verificação da possibilidade de armazenar artefatos ou dependências entre execuções, reduzindo o tempo dos builds subsequentes.
- **Integração com notificações e alertas:** Facilidade de configurar alertas em canais como e-mail, Slack, MS Teams ou outros serviços externos.

Essa análise funcional fornecerá uma visão comparativa inicial sobre a flexibilidade, extensibilidade e facilidade de adoção de cada ferramenta, complementando os resultados obtidos posteriormente nos testes práticos com as *pipelines*.

5.2 Estrutura do Estudo

A metodologia adotada consistirá na implementação de *pipelines* semelhantes nas três plataformas de CI/CD selecionadas. Cada *pipeline* será estruturada para executar as mesmas etapas, assegurando condições equivalentes de comparação entre as ferramentas. As etapas padronizadas contemplarão a restauração das dependências do projeto, a compilação do código-fonte, a execução de testes automatizados utilizando o framework xUnit, a criação da imagem Docker da aplicação e, por fim, a publicação dessa imagem no Docker Hub.

5.3 Procedimento de Implementação

Para garantir equidade na comparação, todas as *pipelines* serão aplicadas sobre o mesmo projeto base, desenvolvido com a tecnologia .NET. As etapas implementadas em cada ferramenta seguirão o seguinte fluxo:

1. **Restaurar dependências:** Utilização do *dotnet restore* para baixar os pacotes via NuGet.
2. **Compilar o projeto:** Execução de *dotnet build* para validar a integridade do código.
3. **Executar testes:** Execução dos testes automatizados com *dotnet test*, utilizando o framework xUnit.

4. **Gerar imagem Docker:** Utilização do Docker CLI para construir a imagem da aplicação.
5. **Verificar vulnerabilidades:** Escaneamento da imagem Docker gerada com a ferramenta Trivy antes do envio ao Docker Hub.
6. **Publicar no Docker Hub:** Autenticação e envio da imagem para um repositório privado, utilizando credenciais armazenadas como variáveis de ambiente seguras em cada plataforma.

Todas as tarefas serão definidas por arquivos YAML nas plataformas GitHub Actions e GitLab CI/CD, e em Groovy (Pipeline Script) no Jenkins.

5.4 Ambiente de Execução

As execuções das pipelines serão realizadas em ambientes controlados, visando garantir a padronização das condições e possibilitar uma comparação justa e precisa entre as ferramentas avaliadas. Cada ferramenta terá seu ambiente configurado da seguinte forma:

- **Jenkins:** Executado localmente em um container Docker, com uma configuração dedicada de 4 GB de RAM e 2 vCPUs, garantindo estabilidade e recursos suficientes durante os testes.
- **GitHub Actions:** Utilizado diretamente no ambiente SaaS fornecido pelo GitHub, com GitHub-hosted runners.
- **GitLab CI/CD:** Executado diretamente na infraestrutura SaaS fornecida pelo GitLab, utilizando o GitLab Shared Runners.

Cada pipeline será executada múltiplas vezes para assegurar a estabilidade dos resultados e minimizar possíveis variações.

5.5 Critérios de Avaliação

As ferramentas serão avaliadas com base nos critérios:

- **Tempo de desenvolvimento inicial:** Tempo real medido desde o início da configuração até a primeira execução bem-sucedida da pipeline, refletindo a praticidade e curva de aprendizado necessária.
- **Tempo médio de execução das pipelines:** Tempo médio aferido após múltiplas execuções completas de todas as etapas padronizadas, indicando eficiência de performance.

- **Facilidade de gerenciamento durante execução:** Avaliação qualitativa da praticidade para monitorar, depurar erros e interagir com as pipelines durante a execução por meio das interfaces das ferramentas.
- **Estabilidade e consistência das execuções:** Verificação da frequência e natureza de eventuais falhas ou erros ocorridos durante múltiplas execuções repetidas.
- **Eficiência no gerenciamento de variáveis e segredos:** Avaliação da praticidade e segurança das plataformas durante o uso efetivo de variáveis e credenciais sensíveis nas execuções reais.
- **Eficácia nas integrações práticas:** Avaliação da capacidade real das plataformas de se integrarem, durante as execuções das pipelines, com ferramentas externas específicas como o Trivy para análise de segurança.

5.6 Tratamento dos Dados

Os dados coletados durante as execuções, incluindo o tempo total gasto na execução das pipelines, registros detalhados de erros (*logs*), feedback gerado pelas ferramentas e esforço necessário para a configuração inicial, serão organizados em tabelas comparativas para facilitar a análise. A avaliação dos resultados seguirá uma abordagem qualitativa e quantitativa, proporcionando uma compreensão abrangente e equilibrada das vantagens, limitações e diferenças práticas entre as plataformas estudadas.

REFERÊNCIAS

- Amazon Web Services. **O que é o DevOps?** [S.l.], 2024. Acessado em 10 de outubro de 2024. Disponível em: <https://aws.amazon.com/pt/devops/what-is-devops/>.
- Amazon Web Services. **AWS, O que significa distribuição contínua?** [S.l.], 2025. Acessado em 1 de fevereiro de 2024. Disponível em: <https://aws.amazon.com/pt/devops/continuous-delivery/>.
- Amazon Web Services. **AWS, O que significa integração contínua?** [S.l.], 2025. Acessado em 1 de fevereiro de 2024. Disponível em: <https://aws.amazon.com/pt/devops/continuous-integration/>.
- Atlassian. **The DevOps lifecycle.** [S.l.], 2025. Acessado em 1 de abril de 2025. Disponível em: <https://www.atlassian.com/devops>.
- BASS, L.; WEBER, I.; ZHU, L. **DevOps: A Software Architect's Perspective.** [S.l.]: Addison-Wesley Professional, 2020.
- bluelight.co. **Best CI/CD Tools.** [S.l.], 2025. Acessado em 10 de janeiro de 2025. Disponível em: <https://bluelight.co/blog/best-ci-cd-tools>.
- DUVALL PAUL M., M. S.; GLOVER, A. **Continuous Integration: Improving Software Quality and Reducing Risk.** [S.l.]: Addison-Wesley Professional, 2007.
- FOWLER, M. **Continuous Integration.** [S.l.], 2024. Acessado em 1 de fevereiro de 2025.
- GitHub, Inc. **Getting Started with GitHub Actions.** [S.l.], 2024. Acessado em 1 de novembro de 2024. Disponível em: <https://github.com/features/actions/getting-started>.
- GitHub, Inc. **GitHub Actions Overview.** [S.l.], 2024. Acessado em 7 de novembro de 2024. Disponível em: <https://resources.github.com/devops/tools/automation/actions/>.
- GitHub, Inc. **Using workflow templates.** [S.l.], 2025. Acessado em 4 de abril de 2025. Disponível em: <https://docs.github.com/en/actions/writing-workflows/using-workflow-templates>.
- GitLab, Inc. **Get started with GitLab CI/CD.** [S.l.], 2024. Acessado em 3 de novembro de 2024. Disponível em: <https://docs.gitlab.com/ee/ci/>.
- GitLab, Inc. **CI/CD steps.** [S.l.], 2025. Acessado em 1 de março de 2025. Disponível em: <https://docs.gitlab.com/ci/steps/>.
- GitLab, Inc. **Gitlab Runners.** [S.l.], 2025. Acessado em 1 de março de 2025. Disponível em: <https://docs.gitlab.com/ci/runners/>.
- GitLab, Inc. **How to choose the right continuous integration tool.** [S.l.], 2025. Acessado em 1 de março de 2025. Disponível em: <https://about.gitlab.com/topics/ci-cd/choose-continuous-integration-tool/>.
- GRACIANO, G. K. K. L. P. **Avaliações de soluções CI/CD: Uma análise comparativa entre GitHub Actions e GitLab CI.** [S.l.]: Fundação Universidade Federal de Mato Grosso do Sul, 2023.
- HUMBLE, J.; FARLEY, D. **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.** [S.l.]: Addison-Wesley Professional, 2010.

Jenkins. **Documentação do Jenkins**. [S.l.], 2024. Acessado em 12 de novembro de 2024. Disponível em: <https://www.jenkins.io/doc/>.

Jenkins. **Suporte de Plugins do Jenkins**. [S.l.], 2024. Acessado em 10 de outubro de 2024. Disponível em: <https://plugins.jenkins.io/>.

Jenkins. **Using Jenkins agents**. [S.l.], 2024. Acessado em 12 de março de 2025. Disponível em: <https://www.jenkins.io/doc/book/using/using-agents/>.

Jenkins. **What is Jenkins Pipeline?** [S.l.], 2024. Acessado em 12 de março de 2025. Disponível em: <https://www.jenkins.io/doc/book/pipeline/>.

Jenkins. **Extending with Shared Libraries**. [S.l.], 2025. Acessado em 12 de março de 2025. Disponível em: <https://www.jenkins.io/doc/book/pipeline/shared-libraries/>.

Jenkins. **Jenkins Core, archiveArtifacts**. [S.l.], 2025. Acessado em 12 de março de 2025. Disponível em: <https://www.jenkins.io/doc/pipeline/steps/core/>.

JetBrains Developer Ecosystem Survey. **JetBrains Developer Ecosystem Survey**. [S.l.], 2023. Acessado em 15 de janeiro de 2025. Disponível em: <https://www.jetbrains.com/lp/devecosystem-2023/>.

LIMA, P. H. O. **Estudo comparativo sobre as funcionalidades de integração contínua e entrega contínua das ferramentas Jenkins e GitLab**. [S.l.]: Repositório Institucional Unichristus, 2021.

Palo Alto Networks. **What Is CI/CD Security?** [S.l.], 2025. Acessado em 15 de maio de 2025. Disponível em: <https://www.paloaltonetworks.com/cyberpedia/what-is-ci-cd-security>.

Red Hat. **O que é YAML?** [S.l.], 2023. Acessado em 1 de fevereiro de 2025. Disponível em: <https://www.redhat.com/pt-br/topics/automation/what-is-yaml>.

RedHat. **O que é CI/CD?** [S.l.], 2025. Acessado em 10 de janeiro de 2025. Disponível em: <https://www.redhat.com/pt-br/topics/devops/what-is-ci-cd>.

RIUNGU-KALLIOSAARI, L.; MÄKINEN, S.; TYRVÄINEN, P. Devops adoption benefits and challenges in practice: A case study. **Software: Practice and Experience**, v. 46, n. 6, p. 779–799, 2016.

SAMANTHA, S. A survey on challenges and benefits towards the adoption of devops approach. **International Journal for Research in Applied Science and Engineering Technology (IJRASET)**, p. 02–06, 2018.

SHAHIN, M.; BABAR, M. A.; ZHU, L. Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. **IEEE Access**, v. 5, p. 3909–3943, 2017.